

Homework #2

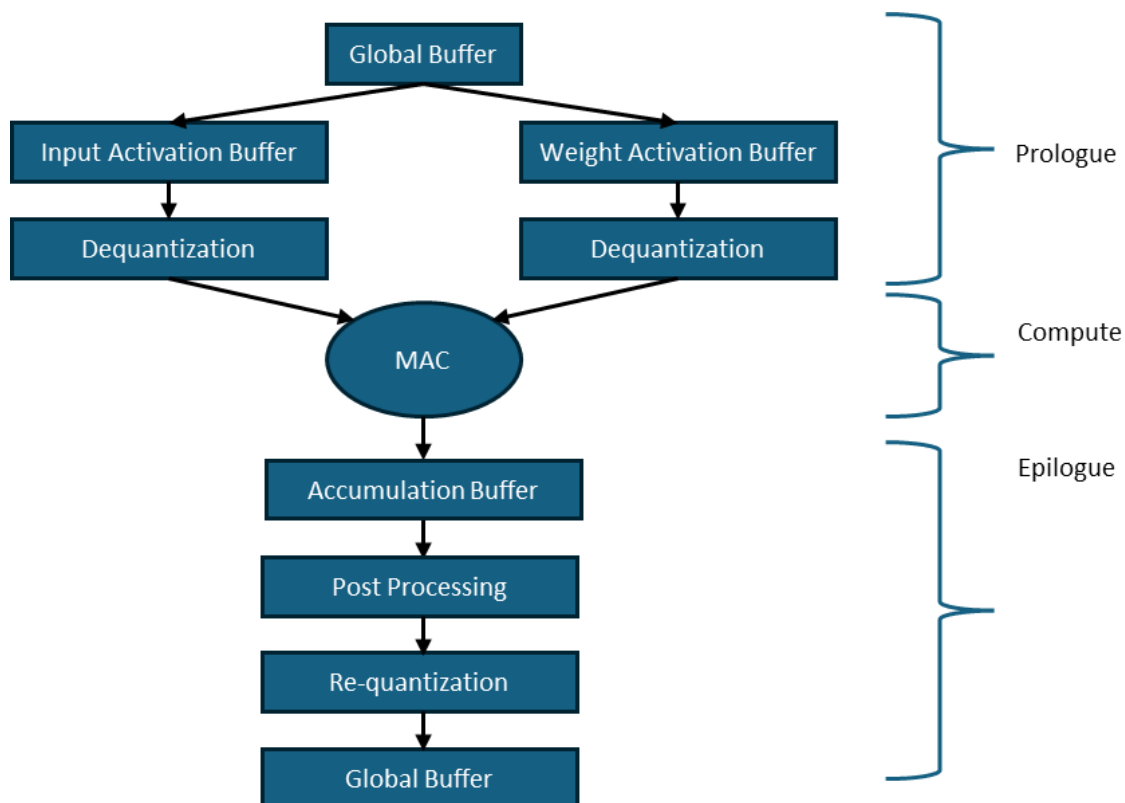
Hardware Overheads of Microscaling

(Due: Sunday, February 8th, 2026, 11:59pm PT)

Total points for this HW = 100 points

The trajectory of hardware acceleration has been defined by a pursuit of computational density and memory bandwidth efficiency. As neural network parameter counts scale the traditional single-precision floating-point format (FP32) has become a primary bottleneck. The silicon area required for 32-bit arithmetic units and the energy consumed moving 32-bit operands across the memory hierarchy are no longer sustainable. This physical reality necessitated the development of lower-precision formats.

The most recent and significant advancement in this domain is the standardization of Microscaling (MX) Formats. This introduces a fundamental shift from scalar quantization (where every number carries its own scale/exponent) to Block Floating Point (BFP) architectures. In an MX system, a contiguous block of elements (typically $k=32$) shares a single high-precision scaling factor, while the individual elements are stored in extremely narrow bit-widths, such as 4-bit integers or 4-bit floating-point numbers (E2M1). **The goal of this assignment is to analyze the architectural implications and trade-offs introduced by the MX Formats.**



In the context of AI accelerator design, the terms "Prologue" and "Epilogue" refer to distinct phases of the execution pipeline for a compute kernel.

- **Prologue:** This phase encompasses the frontend of the operation. It involves the generation of memory addresses, the fetching of input & weight activations from the multi-level memory hierarchy, dequantization and the movement of this data into the register files of the compute units.
- **Compute Kernel:** This is the dense arithmetic phase, executed by Systolic Arrays. Here, Multiply-Accumulate (MAC) operations are performed in higher precision to preserve numerical fidelity during the summation.
- **Epilogue:** This phase occurs after the accumulation is complete but before the data is written back to global memory. In standard FP32 architectures, the epilogue might simply involve rounding. However, in MX architectures, the epilogue becomes a computationally non-trivial stage. The hardware must inspect a block of 32 high-precision partial sums, determine the maximum absolute value to calculate the new shared exponent (E_{block}), and then dynamically shift and round all 32 values to fit into the narrow destination format (e.g., MXFP4/MXFP8). In addition to the requantization, the epilogue often incorporates post-processing operations, including vector-level transformations such as activation functions (e.g., ReLU, sigmoid, RMS), as well as scaling and truncation.

Recommended Reading:

- [Floating-Point 8: An Introduction to Efficient, Lower-Precision AI Training | NVIDIA Technical Blog](#)

PART A: Estimating Epilogue Latency

(15 + 7.5 + 7.5 Points)

You are designing a MXFP8 accelerator, where the data consists of E4M3 elements shared across a block with E8M0 scale factor. The hardware processes data in blocks (of size $k = 32$). The Epilogue unit receives 32 high-precision partial sums from the MAC unit & compresses them into a single MX block; it must find the maximum absolute value among all 32 numbers to determine the new shared exponent, shift the mantissas, and round the values.

- Determine the configuration to find the maximum of the elements in a block to minimize the latency? How many 2-input comparators do you need for the same and what is the latency for this configuration? (Assume a 2-input comparator has a latency of a single clock cycle.)
- Modern AI accelerators use wide vector units. If your vector unit is 128 lanes wide (processing 128 elements in parallel), how many total 2-input comparators are required? (Hint: Does a 128-wide vector share one scale, or is it composed of multiple independent MX blocks.)
- After the maximum is found, assuming the hardware needs 1 additional cycle to calculate the scale, shift mantissas, and round the values. Calculate the Total Epilogue Latency (from receiving partial sums to having a valid MX block ready for memory).

PART B: Estimating Prologue & Compute Latency

(10 Points)

Prior to the epilogue stage, the accelerator fetches operands during the prologue and performs the dot-product computation during the compute stage. The accelerator supports only FP16 operations, while MXFP8 is used for improved memory efficiency. To enable computation, MXFP8 operands are converted to FP16 by inserting a dequantization stage before the compute stage.

Calculate the total latency for a partial sum to reach the epilogue stage, given the following component latencies:

- SRAM Read: 2 cycles
- Dequantization: 3 cycles
- Fused Multiple-Add: 1 cycle

PART C: Comparative Analysis and Trade-Offs

(25 + 20 Points)

- Compare the Total Epilogue Latency with the (Prologue + Compute) Latency for $k = 32$. Are the latencies balanced? Also, compare latencies & memory overhead with block sizes $k = 8, 16, 32$ & 64. Comment on the trade-offs here when increasing the block size?

Block Size (k)	Epilogue Latency	Number of cycle stalls per iteration	Memory Overhead
8			
16			
32			
64			

Hint: In the context of Microscaling (MX) Formats, memory overhead (or Scale Tax) refers to the extra bits added to the raw data to store the shared scale factor. (Data consists of E4M3 elements shared across a block with E8M0 scale factor.)

- b. You are a computer architect at a startup. Your compiler team decided a block size $k = 128$ to save power on Memory transfers. How many cycle stalls would this block size cause on every iteration? Propose two different hardware solutions to fix this stall. Briefly discuss the Area overhead of each. (Hint : One should focus on buffering, and the other should focus on parallelism.)

PART D: Alternative Design Choices

(15 Points)

For the accelerator architecture described in Parts A and B, assume you are free to redesign the system from the ground up with the explicit goal of optimizing latency and silicon area when using microscaled (MX) number formats. What architectural or microarchitectural design choices would you make to best support MX computation?